Not Yet Quantum-Ready: Shortcomings of Open-Source Static Source-Code Analyzers for Post-Quantum Cryptography Compliance, and a Roadmap for Improvement

Abstract—Migration mandates now require organizations to inventory quantum-vulnerable cryptography, and United States federal guidance explicitly names static application security testing as a discovery method. This paper examines whether open-source static source-code analyzers can support post-quantum cryptography (PQC) compliance determinations. We decompose "PQC compliance" into seven obligations, characterize three families of analyzers, and run ten tools in eleven configurations against a micro-benchmark of 35 Python and 7 Java cases. General-purpose SAST engines reported at most 2 of 17 quantum-vulnerable Python usages, and only through classical weak-key or deprecated-library rules. PQC-branded scanners reported between 6 and 13 of 17 on code lines but failed on semantics-preserving variants: no tool resolved a reflectively loaded RSA key generator, and only one found an algorithm selected in a YAML file. No tool recognized ML-KEM or ML-DSA used through pyca/cryptography's native API, four analyzers misclassified ML-DSA as classical DSA, CodeQL's standard Java suite flagged ML-KEM and ML-DSA as potentially weak, and every tool that saw a hybrid reported its X25519 component as a stand-alone vulnerability. No tool checked KEM construction, policy profiles, reachability or size-based migration blockers. We synthesize ten shortcomings and propose a reference architecture that separates evidence from dated policy judgment, maintains tested API models, fuses evidence across artifacts, adds PQC-specific construction and agility checks, and reports coverage explicitly.

Index Terms—post-quantum cryptography, static analysis, cryptographic inventory, CBOM, crypto-agility, compliance, ML-KEM, ML-DSA.

I. Introduction

Public-key cryptography built on integer factorization and discrete logarithms, which covers RSA, finite-field Diffie-Hellman and their elliptic-curve counterparts, is broken in polynomial time by Shor's algorithm running on a sufficiently large fault-tolerant quantum computer [1]. Resource estimates for such a cryptographically relevant quantum computer (CRQC) have fallen steadily. Gidney and Ekerå estimated in 2021 that 2048-bit RSA could be factored in about eight hours with twenty million noisy qubits [2], and Gidney revised the requirement in 2025 to fewer than one million noisy qubits running for under a week [3]. Because ciphertext recorded today can be decrypted once such a machine exists, long-lived confidential data is already exposed to harvest-now-decrypt-later collection. Mosca's inequality states the planning consequence: if the time data must stay secret plus the time needed to migrate exceeds the time until a CRQC arrives, the organization is already late [4].

Standards and mandates have turned this risk into deadlines. NIST published ML-KEM, ML-DSA and SLH-DSA as FIPS 203, 204 and 205 in August 2024 [5]–[7], and its draft transition report proposes deprecating quantum-vulnerable public-key algorithms after 2030 and disallowing them after 2035 [8]. In June 2026, OMB memorandum M-26-15 directed United States civilian agencies to mitigate as much quantum risk as feasible by 31 December 2030, to migrate key establishment for priority systems in 2028–2030 and signatures in 2031, to finish by 2035, and to submit migration plans within 120 days that state which methodologies and automated tools produced their cryptographic inventories [9]. Canada's Cyber Centre roadmap (ITSM.40.001) set April 2026 for initial departmental migration plans, annual progress reporting thereafter, the end of 2031 for high-priority systems and the end of 2035 for all remaining systems, and it defines completion as quantum-vulnerable algorithms being disabled, isolated or tunnelled rather than merely PQC being supported [10]. National security systems in the United States follow the NSA's Commercial National Security Algorithm Suite 2.0 (CNSA 2.0), which narrows the acceptable choices further [11].

Every one of these programs begins with discovery, and discovery increasingly means automation. M-26-15 names static application security testing (SAST) "to find cryptographic functions in code" alongside software composition analysis and network scanning as feeders of a central Cryptographic Bill of Materials (CBOM) [9]. The Canadian roadmap likewise lists source code among the artifacts that discovery tools scan, while cautioning that discovery should combine multiple methodologies [10]. A natural first step for many organizations is therefore to reuse what already runs in continuous integration, namely open-source static analyzers such as Semgrep, CodeQL and Bandit, or to adopt newer PQC-oriented tools such as the sonar-cryptography plugin and CBOMkit. Their findings can easily end up cited as evidence of "PQC compliance" in migration plans, audits and vendor attestations.

This paper argues that such evidence is weaker than it appears. Static crypto-misuse detectors have already been shown to miss semantically equivalent variants of the very misuses they claim to detect [12]. The PQC setting adds problems that classical misuse detection never had to solve. The target is a moving, jurisdiction-specific and time-dependent policy rather than a fixed list of broken primitives. Compliance requires positive evidence that approved algorithms and parameter sets are present and used correctly, not only the absence of weak ones. Many of the most consequential choices, such as TLS key-exchange groups, library defaults, provider configuration and certificates, live outside the source code an analyzer reads. Ironically, the crypto-agility practices that migration guidance recommends, such as moving algorithm selection into external configuration [9], make a source-only analyzer less able to see what a deployed system will actually do.

We organize the study around three research questions. RQ1 asks what open-source static source-code analyzers actually check when they are used to assess PQC compliance. RQ2 asks where and why they fail, drawing on published evaluations, tool documentation and rule sets, and a small controlled micro-benchmark. RQ3 asks how they can be improved so that their output can credibly support a compliance determination.

The paper makes five contributions. First, it decomposes "PQC compliance" into distinct obligations and identifies which of them a static source-code analyzer can decide in principle (Section II). Second, it characterizes three families of open-source analyzers,

namely PQC-oriented discovery tools, general-purpose SAST engines with cryptographic rules, and academic crypto-misuse detectors, according to what each actually models (Section III). Third, it reports an illustrative and reproducible micro-benchmark of Python programs that exercise migration-relevant coding patterns, executed against several widely used open-source analyzers (Section IV). Fourth, it synthesizes the evidence into a taxonomy of shortcomings (Section V). Fifth, it proposes a reference architecture and concrete techniques for PQC-aware static analysis, followed by a research agenda (Sections VI and VII).

II. Background

A. Post-quantum standards and transition policy

Shor's algorithm breaks the factoring and discrete-logarithm problems beneath RSA, DSA, finite-field and elliptic-curve Diffie-Hellman, ECDSA and EdDSA [1], whereas Grover's search yields only a quadratic speed-up against symmetric ciphers and hash functions [13], which larger keys and digests absorb. Migration therefore concentrates on public-key key establishment and digital signatures. NIST's first standards define ML-KEM for key encapsulation with the parameter sets ML-KEM-512, ML-KEM-768 and ML-KEM-1024 [5], ML-DSA for signatures with ML-DSA-44, ML-DSA-65 and ML-DSA-87 [6], and the stateless hash-based SLH-DSA [7]. They complement the stateful hash-based LMS and XMSS schemes of SP 800-208 [14]. A FALCON-based signature standard (FN-DSA, to be FIPS 206) and a standard based on the code-based KEM HQC, selected in March 2025 as a backup to ML-KEM, remain in the pipeline [15]. SP 800-227, finalized in September 2025, adds normative guidance on how KEMs are used, including the handling of ephemeral and static key pairs, key confirmation and approved combiners for multi-algorithm (hybrid) KEMs [16].

Transition policy is layered on top of these algorithm standards, and it is neither uniform nor static. NIST's draft IR 8547 distinguishes security strengths: quantum-vulnerable schemes at the 112-bit level, such as RSA-2048, are proposed for deprecation after 2030, and all quantum-vulnerable schemes for disallowance after 2035 [8]. CNSA 2.0 admits only ML-KEM-1024 and ML-DSA-87 among the general-purpose PQC parameter sets, requires AES-256 and SHA-384 or SHA-512, reserves LMS and XMSS for software and firmware signing, and sets category-specific dates after which only CNSA 2.0 algorithms may be used [11]; NSA's FAQ updates restrict usage further, for example by excluding the pre-hash variant of ML-DSA from national security systems [17]. M-26-15 enumerates the quantum-vulnerable algorithms that agencies must target and advises treating any asymmetric algorithm not definitively known to be quantum-resistant as vulnerable [9]. Canada's Cyber Centre defines acceptable algorithms for unclassified, Protected A and Protected B information in ITSP.40.111 and asks procurements to require cryptographic modules certified by the Cryptographic Module Validation Program [10], [18].

The same line of code can therefore be compliant under one profile and non-compliant under another. ML-KEM-768 satisfies NIST guidance but not CNSA 2.0, and AES-128 is acceptable to NIST but not to CNSA 2.0. RSA-2048 and RSA-3072 signatures look identical to a classical key-size rule, yet the IR 8547 draft proposes deprecating the former after 2030 while allowing the latter until 2035 [8]. Hybrid constructions add a further layer: IR 8547 accommodates them during the transition [8], M-26-15 describes the TLS 1.3 hybrid key exchange that sends both an X25519 and an ML-KEM-768 key share and combines the two secrets [9], and SP 800-227 specifies approved combiners [16]. A compliance assessment must recognize that X25519 inside an approved hybrid is not the same finding as X25519 on its own.

B. What "PQC compliance" requires

Discussions of PQC compliance often collapse several distinct obligations into a single verdict. We separate seven, because they differ sharply in how much a static source-code analyzer can establish.

The first obligation is inventory completeness: every use of quantum-vulnerable public-key cryptography, whether for key establishment, signatures, certificates or protocol negotiation, must be found, located and attributed to a purpose [9], [10]. The second is algorithm and parameter conformance with a named policy profile at a named date, as described above. The third is construction correctness. Approved algorithms must be used as their standards require, which for PQC includes deriving keys from KEM shared secrets through approved methods, handling ephemeral and static KEM keys as SP 800-227 prescribes, building hybrids with approved combiners [16], selecting the intended ML-DSA variant, and never reusing the state of a stateful hash-based signature key [14]. The fourth is implementation provenance, meaning that the code performing an operation comes from a validated module wherever policy requires one [10]. The fifth is crypto-agility, the ability to replace algorithms through configuration or negotiation rather than code changes, which M-26-15 asks every migrated system to provide and NIST CSWP 39 elaborates [9], [19]. The sixth is prioritization context, chiefly how long protected data must remain confidential and how critical the system is, which together determine exposure to harvest-now-decrypt-later collection [4], [10]. The seventh is deployment enforcement: Canada's roadmap counts a system as migrated only when quantum-vulnerable algorithms are disabled, isolated or tunnelled [10].

A static analyzer that reads only source code can, in principle, contribute evidence to the first, second, third and fifth obligations. Even there it is limited to cryptography invoked from the analyzed code, through APIs the analyzer models, with parameters it can resolve. It cannot by itself establish provenance, which depends on build artifacts and validation records, nor prioritization, which depends on data classification, nor enforcement, which depends on deployed configuration and negotiated behavior. This bound is not a defect of any particular tool, but it determines what a tool's silence can honestly mean.

C. Cryptographic inventories and the CBOM

The CycloneDX Cryptography Bill of Materials (CBOM), introduced in CycloneDX 1.6 in April 2024 and standardized through Ecma International as ECMA-424, represents algorithms, keys, certificates, protocols and related material as typed cryptographic-asset components. Each component carries properties such as its primitive, parameter-set identifier, object identifier and estimated NIST quantum security level, together with evidence of where it occurs [20]. Version 1.7, released in October 2025, added a standardized

registry of algorithm families to improve consistency across tools [20]. M-26-15 envisions a central, continuously updated CBOM fed by SCA, SAST, DAST and network scanning [9]. NIST's NCCoE, whose preliminary SP 1800-38B defines a functional test plan for discovery tools, likewise treats discovery as a multi-tool activity rather than a single scan [21]. A CBOM is a data model, however, and not an assurance case: it records what a tool reported, not what the tool was able to see.

D. Static-analysis techniques behind crypto detectors

Crypto detectors span a spectrum of analysis depth. At the shallow end, keyword and regular-expression scanners such as Wind River's Crypto Detector search files for algorithm names, library identifiers and API strings [22]. Syntactic pattern matchers such as Bandit for Python and Semgrep for many languages match abstract-syntax-tree patterns of known API calls and literal arguments, with Semgrep adding intra-procedural constant propagation [23], [24]. Deeper engines translate a program into a queryable relational representation and support inter-procedural data-flow and taint tracking; CodeQL, built on the QL language, is the most widely used example, with open-source query libraries running on an engine distributed under GitHub's own terms [25]. Specification-based analyzers such as CogniCrypt's CrySL describe the correct usage of each cryptographic class as a typestate machine with constraints and check programs against it [26], building on typestate [27], interprocedural data-flow frameworks such as IFDS [28] and abstract interpretation [29].

Every engine in this spectrum faces the same theoretical ceiling. Non-trivial semantic properties of programs are undecidable [30], so practical analyzers are "soundy": they handle reflection, dynamic loading and native code through unsound assumptions that are rarely surfaced in their output [31]. The consequence for compliance is that a clean report never proves absence. It only reports what fell inside the analyzer's model.

III. Open-Source Analyzers Used for PQC Assessment

The analyzers that practitioners can apply to PQC assessment without a commercial license fall into three families: PQC-oriented discovery tools that emit a cryptographic inventory, general-purpose SAST engines whose rule sets include cryptographic checks, and academic crypto-misuse detectors. This section characterizes what each family models, using documentation, rule sets and source code inspected in October 2026; Section IV then measures behavior.

A. PQC-oriented discovery tools

The most mature open-source discovery tool is the Sonar Cryptography Plugin, also called CBOMkit-hyperion, which IBM Research donated to the CBOMkit project now hosted under the Post-Quantum Cryptography Alliance [32], [33]. It applies language-specific detection rules and writes a CycloneDX CBOM. Its documentation claims full coverage of the Java Cryptography Architecture (JCA) and the BouncyCastle lightweight API, of pyca/cryptography for Python and of Go's standard crypto packages except crypto/x509, partial coverage of golang.org/x/crypto, and early C# support without cross-method tracking [32]. Two limits matter for compliance use. First, the plugin runs inside a SonarQube instance, which raises the cost of adoption and prevented us from executing it in our sandbox [32]. Second, its coverage claims are relative to a library snapshot. In the repository state of 8 October 2026, the Java, Go and C# modules model ML-KEM and ML-DSA, but the Python module contains no ML-KEM or ML-DSA rules, although pyca/cryptography 50.0.2 ships native mlkem and mldsa modules [36]. A claim of complete library coverage can therefore coexist with blindness to that library's post-quantum API. The companion CBOMkit-theia inspects container images and directories for certificates, keys, java.security and OpenSSL configuration, which partly covers what source scanning cannot see [33].

Since 2025, a second wave of small PQC-branded scanners has appeared on PyPI. We examined six that install as command-line tools under OSI licenses: qproof [37], cryptohound [38], pqc-audit [39], pqready [40], crypto-scanner [41] and pqc-posture [42]. They advertise quantum-vulnerability classification, migration advice, letter grades or readiness scores, and in several cases CycloneDX CBOM or SARIF output. Their engines range from regular expressions and tree-sitter matching (qproof, crypto-scanner) through Semgrep rule packs (cryptohound) to AST visitors (pqc-audit). Their packaging maturity is uneven. The published pqc-posture 0.4.0 command fails at start-up because a module is missing from the package, cryptohound 0.1.0 crashed in a fresh environment because its Semgrep dependency is incompatible with the newest release of a transitive dependency, and cryptohound's metadata points to a placeholder repository. Their classification schemes also embed fixed opinions: qproof's documentation labels MD5 and SHA-1 as quantum-vulnerable and SHA-256 as safe [37], merging classical weakness with quantum risk and ignoring the SHA-384 floor of CNSA 2.0.

Wind River's Crypto Detector predates the PQC transition. It scans source trees with keyword lists and an API-finder method and emits a JSON evidence report [22]. Its algorithm lists contain no post-quantum algorithms, its most recent commit dates from January 2020, and on Python 3.12 its API-finder method crashed, leaving only the keyword method usable.

B. General-purpose SAST with cryptographic rules

Bandit [23] and Semgrep [24] ship cryptographic rules designed around classical weakness. Bandit checks for weak hashes and ciphers, deprecated PyCrypto imports, and RSA or DSA keys shorter than 2048 bits. The Semgrep community rule repository, distributed under the Semgrep Rules License rather than an OSI license [35], contains Python rules for insufficient RSA, DSA and EC key sizes, weak hashes and ciphers, ECB and unauthenticated modes. In the revision we fetched on 15 October 2026, no rule file mentioned quantum risk, ML-KEM, Kyber, ML-DSA or Dilithium. Such rules are useful for classical hygiene, but a key-size rule is structurally unable to flag RSA-3072 or P-384, which are classically strong and quantum-vulnerable.

CodeQL [25] combines open-source query libraries with an engine distributed under GitHub's own terms. Its standard security suites contain weak-key and weak-algorithm queries for Python and Java. GitHub has also been developing an experimental "quantum"

library that models cryptographic operations as a graph suitable for CBOM generation [34]. In CodeQL CLI 2.27.2, that work ships as 34 experimental Java queries, including inventory "slices" of known asymmetric algorithms, plus a single CBOM-graph printer for C and C++, and nothing for Python. Its shared algorithm model enumerates DSA, ECDSA, EdDSA and HSS-LMS signatures and Kyber and FrodoKEM key encapsulation; it has no ML-DSA, SLH-DSA or FN-DSA type and names the standardized KEM by its pre-standard name.

C. Academic crypto-misuse detectors

A decade of research on cryptographic API misuse produced CryptoLint for Android [43], CryptoGuard for large Java projects [44], CogniCrypt and its CrySL specification language [26] and LICMA for Python [47], together with benchmarks such as CryptoAPI-Bench [45] and CamBench [46]. These are the most semantically sophisticated open analyzers available, built on program slicing, typestate and data-flow analysis, and specification languages such as CrySL are directly reusable for PQC. Their target, however, is misuse of classical primitives, such as constant keys, static initialization vectors, ECB mode or low iteration counts, rather than inventory or policy conformance. MASC's mutation-based evaluation showed that detectors of this kind miss semantics-preserving variants of misuses they claim to catch [12], a warning that applies directly to PQC inventory tools built on similar matching techniques.

D. What the tools actually check

Read against the seven obligations of Section II-B, the landscape is narrow. General-purpose SAST checks classical strength and therefore says nothing about quantum vulnerability at classically adequate parameters. PQC-oriented tools address inventory completeness and a coarse form of algorithm conformance, usually a vulnerable-or-safe label tied to one implicit policy, and some emit CBOMs. Apart from qproof's user-managed policy files, which we did not evaluate, none of the tools exposes a selectable, dated policy profile, and none models construction correctness for KEMs, hybrids or stateful signatures, implementation provenance, crypto-agility, or deployment context. Section IV tests how well even the narrow inventory task is performed.

IV. An Illustrative Micro-Benchmark

We built a small benchmark to observe how the analyzers of Section III behave on code that a PQC migration actually encounters. It is illustrative rather than representative: its value lies in controlled contrasts between semantically equivalent programs, not in population-level detection rates.

A. Benchmark design

The benchmark contains 35 Python cases and 7 Java cases, catalogued in Table A2 of the Appendix. Each Python case is a short, self-contained module; the Java cases use the JCA, including the ML-KEM and ML-DSA support added in JDK 24 [48], [49]. Eight canonical cases use quantum-vulnerable primitives directly through pyca/cryptography or PyCryptodome: RSA-2048, RSA-1024, ECDSA over P-256, X25519, Ed25519, finite-field Diffie-Hellman, RSA-OAEP with a loaded public key, and RSA-3072. Five variants preserve the semantics of a canonical case while changing its form, in the spirit of MASC's mutation operators [12]. They use an aliased import with positional arguments, a key-generation helper that receives the algorithm module as a parameter, an algorithm family read from a YAML file, a curve named by an environment variable, and a module resolved by reflection from a string built at run time. Six cases reach cryptography through third-party libraries, protocols or services: an RS256 JSON Web Token, Paramiko SSH keys, a default TLS context, the OpenSSL command-line tool, an HTTPS request through requests, and a cloud KMS key created with an RSA key specification.

Eight post-quantum cases use ML-KEM-768 and ML-DSA-65 through liboqs and through pyca/cryptography's native API, ML-KEM-512, two X25519 plus ML-KEM-768 hybrids, and an ML-KEM misuse in which the shared secret becomes an AES key without key derivation and one KEM key pair serves every session. Seven controls test context and precision: a file whose only references to RSA, ECDSA and Diffie-Hellman are comments and strings, an error-correcting-code module with identifiers named ecc, dh and dsa, SHA-1 marked usedforsecurity=False, unreachable RSA code, an RSA test fixture, AES-128-GCM and SHA-256. A final case contains no cryptography but hard-codes 256-byte signatures and 294-byte public keys into a wire format, a migration blocker because an ML-DSA-65 signature is 3,309 bytes [6]. The Java cases mirror the RSA, ECDSA and X25519 usages, add JCA ML-KEM-768 and ML-DSA-65, take the algorithm from a properties file, and open a TLS 1.3 socket whose key-exchange groups are left to the platform.

Ground truth was fixed before any tool ran. Twenty-one cases, 17 in Python and 4 in Java, contain a quantum-vulnerable usage that a complete inventory must report. Seven contain a stand-alone post-quantum usage that a compliance assessment should credit. The remaining cases probe hybrids, policy dependence, construction defects, context, transport defaults and agility.

B. Tools and procedure

We ran Bandit 1.9.4; Semgrep 1.181.0 with the community Python and Java rules; CodeQL 2.27.2 with the Python and Java security-extended suites and, for Java, five experimental quantum queries; Wind River Crypto Detector with its keyword method; qproof 0.4.0; cryptohound 0.1.0; pqc-audit 0.4.1; pqready 0.2.2; crypto-scanner 0.1.2; and pqc-posture 0.4.0. This gives eleven configurations of ten tools. Java was analyzed with CodeQL's build-free extraction. Two tools needed workarounds that a typical user might not find: cryptohound ran only after a working Semgrep was placed first on the search path, and pqc-posture ran only through its Python API. All runs used default settings in a Linux container on 15 October 2026.

Each tool's native output, whether SARIF, CycloneDX CBOM, JSON or text, was normalized to file, line and label, and labels were mapped to algorithm families. A case counts as detected when a tool reports the expected family in one of the case's files. Because several scanners match text, we report recall twice, once on all findings and once on findings located on code lines only, excluding

comments and docstrings, and we treat the code-line figure as primary. Python-only tools are scored on the 17 Python cases. The benchmark, raw outputs and scoring scripts accompany this paper.

C. Inventory recall

The three general-purpose SAST engines found almost nothing. On the 17 Python quantum-vulnerable cases, Bandit reported 2, Semgrep 1 and CodeQL 1, and every hit came from a classical rule. All three flagged RSA-1024 as breakable, and Bandit's second hit was its deprecated-PyCrypto rule firing on a PyCryptodome import that happened to name the RSA module. RSA-2048, RSA-3072, ECDSA over P-256, X25519, Ed25519 and finite-field Diffie-Hellman were invisible to all three. On Java, Semgrep and CodeQL's standard suite reported none of the four quantum-vulnerable usages.

The PQC-branded scanners did markedly better on canonical code and markedly worse as soon as the code moved away from it. On code lines, qproof found 13 of the 17 Python cases, Crypto Detector 12, cryptohound 11, pqc-audit and crypto-scanner 10 each, pqc-posture 9 and pqready 6. Cryptohound found all eight canonical cases with no false positives, yet only two of the five variants. Comment matching inflated two tools: qproof's all-findings recall was 15 and Crypto Detector's 13, because their only evidence for the Diffie-Hellman or RSA-OAEP cases was a docstring.

The variants exposed the matching techniques underneath. No tool identified RSA in the reflective variant; Crypto Detector reported only generic cryptography keywords and Semgrep flagged the dynamic import as a code-injection risk. Only Crypto Detector found RSA in the YAML-configured variant, because its keyword scan reads YAML, while pqready labelled the YAML file as containing a "weak cipher identifier." The injected-module variant was found only by tools that key on the import statement, namely Crypto Detector, qproof, pqready and crypto-scanner. The two tools that match key-generation call sites, cryptohound and pqc-audit, missed it, as did pqc-posture. In Java, CodeQL's experimental quantum queries found all four usages, including RSA in the case whose algorithm names come from a properties file, which made CodeQL the only analyzer in the study to resolve configuration semantically. Third-party paths were uneven: qproof found all four of the JWT, Paramiko, OpenSSL CLI and cloud KMS cases, Crypto Detector three, and the other PQC scanners at most two.

D. Post-quantum recognition and hybrids

Crediting compliant code proved harder than finding vulnerable code. Of the seven stand-alone post-quantum cases, crypto-scanner labelled five as generically post-quantum and pqc-audit recognized three, all through liboqs algorithm strings. No tool recognized ML-KEM or ML-DSA used through pyca/cryptography's native classes. Four analyzers misread ML-DSA as classical DSA: qproof and Crypto Detector on the liboqs, pyca and JCA cases, crypto-scanner on the JCA case while simultaneously labelling it post-quantum, and CodeQL's experimental inventory, which also classified ML-KEM-768 as a key-agreement algorithm. CodeQL's standard Java suite went further and reported ML-KEM-768 and ML-DSA as potentially weak cryptographic algorithms, while saying nothing about RSA-2048, ECDSA over P-256 or X25519 in the same code base.

Hybrids were never recognized as such. Each of the five tools that saw the X25519 half of the hybrid cases reported it as a stand-alone quantum-vulnerable exchange. In the hybrid built from pyca/cryptography primitives, the ML-KEM half was invisible to every tool, so a construction following the hybrid pattern described in M-26-15 [9] looked like a purely classical key exchange.

E. Policy, construction, context and agility

No tool distinguished policy profiles. ML-KEM-512 was reported as acceptable post-quantum cryptography by the two tools that saw it, with no indication that it falls below CNSA 2.0. AES-128-GCM was flagged as a Grover-related concern by qproof and pqc-posture, matched as a keyword by Crypto Detector, and ignored by the rest. SHA-256 received contradictory verdicts: qproof marked it as needing no change, while crypto-scanner rated it a medium-risk finding. Six tools flagged the non-security SHA-1 case, four of them PQC-branded scanners that report it within their quantum-risk output, whereas Bandit and CodeQL honored usedforsecurity=False. No tool reported the ML-KEM misuse, and pqc-audit listed the misused ML-KEM-768 as an informational post-quantum asset.

Context was largely ignored. Seven tools reported the unreachable RSA code without qualification, and six reported the RSA test fixture. Only pqc-posture lowered the fixture's severity because it lives in a test file, and cryptohound omitted it, consistent with excluding test directories. On code lines, qproof and Crypto Detector raised false positives on the comment-and-string file and on the error-correcting-code module, where variables named dsa and dh were read as algorithms.

Transport defaults were mostly invisible. Only CodeQL and Crypto Detector flagged the default TLS context, CodeQL because it permits TLS 1.0, a classical issue, and no tool flagged the HTTPS request made through requests. For the Java TLS 1.3 socket, qproof warned that its key exchange is quantum-vulnerable, which may or may not hold depending on the platform's configured groups, while pqc-posture reported it as TLS 1.0 or 1.1. No tool noticed the fixed 256-byte signature framing; qproof and Crypto Detector instead reported RSA in that file because its docstring mentions RSA-2048.

F. Summary

Table A1 in the Appendix consolidates the per-tool results. The pattern is consistent across families: analyzers find what they were written to pattern-match, stay silent about what they cannot model, and do not say which of the two a given silence represents.

V. A Taxonomy of Shortcomings

The evidence of Sections III and IV, read together with the literature, points to ten recurring shortcomings. They are not independent. Several follow from one design decision shared by almost every tool we examined: detection and judgment are fused

into a single rule, and only positive findings are reported.

S1. Policy-model mismatch

General-purpose rules encode classical strength. A key-size rule fires on RSA-1024 and stays silent on RSA-2048, RSA-3072 and every elliptic-curve scheme, so a clean result from Bandit, Semgrep or CodeQL's standard suites carries no information about quantum exposure. PQC-branded scanners replace that model with another fixed one, usually a vulnerable, partial or safe label, and the labels disagree across tools, as the contradictory SHA-256 verdicts showed. None encodes the fact that compliance is relative to a profile and a date: ML-KEM-512 is compliant under FIPS 203 and non-compliant under CNSA 2.0 [5], [11], and IR 8547 proposes different dates for RSA-2048 and RSA-3072 [8]. Several scanners also conflate classical weakness with quantum risk by scoring MD5 or a non-security SHA-1 as quantum exposure, which inflates risk scores with findings that a PQC migration does not address.

S2. Stale and incomplete API models

Detection is only as current as the analyzer's model of each cryptographic library, and libraries are moving faster than models. pyca/cryptography and the JDK now expose ML-KEM and ML-DSA [36], [48], [49]. Yet no analyzer we ran recognized pyca's native classes, sonar-cryptography's Python rules omit them despite a full-coverage claim, and CodeQL's experimental model lacks ML-DSA, SLH-DSA and FN-DSA types. Stale models fail in both directions. They hide compliant code, so migration progress cannot be measured, and they mislabel it: four analyzers read ML-DSA as classical DSA, and CodeQL's standard Java suite reports ML-KEM and ML-DSA as potentially weak because its allow-list predates them. A developer who follows that advice would move away from the very algorithms that mandates require.

S3. Brittleness to semantics-preserving variation

Most PQC scanners match syntax, namely an import, a call site or an algorithm string. Semantically equivalent programs that change that syntax defeat them, exactly as MASC found for classical misuse detectors [12]. In our benchmark the reflective variant evaded all eleven configurations, and the injected-module variant split the tools neatly by whether they keyed on imports or on calls. Reflection, dynamic loading and dependency injection are known weak points of static analysis in general [31], [52]. The specific problem here is that the tools neither model these constructs nor report that a cryptography-capable call could not be resolved.

S4. Configuration-, default- and service-driven cryptography

Many decisions that determine quantum exposure are not written in application source. TLS groups come from library defaults and system configuration, cloud KMS key specifications are request parameters, OpenSSL command lines and configuration files select algorithms, and crypto-agile designs move the choice into YAML or properties files. Migration guidance actively encourages the last of these [9], which creates an agility paradox: the more faithfully a system follows agility advice, the less a source-only analyzer can see. Only one tool resolved a configured algorithm semantically, and only for a Java properties file. The TLS cases were either missed or answered with guesses, including one tool that reported a TLS 1.3 socket as TLS 1.0 or 1.1.

S5. Dependency and binary blindness

Source analyzers see the repository, not the system. The HTTPS request through requests, which reaches OpenSSL through urllib3 and Python's ssl module, produced no PQC finding from any tool. None of the source scanners links its findings to the cryptographic capabilities of third-party packages, vendored code, native extensions or firmware. A CBOM assembled from such scans records only what was seen, whereas M-26-15's combination of SCA, SAST, DAST and network scanning [9] and Canada's call for multiple discovery methodologies [10] both presuppose that results from different vantage points can be joined, which current tools do not support.

S6. Context blindness

Compliance work needs to know whether a usage is reachable, whether it runs in production or only in tests, what it is for, and how long the data it protects must stay secret [4], [10]. Nearly all tools reported unreachable and test-only RSA with the same severity as production code, one labelled an RSA signing operation as key exchange, and text-matching scanners raised false positives on comments, strings and identifiers such as dsa and dh. False positives are not cosmetic. Developers abandon analyzers whose warnings they cannot trust [50], [51], and in a migration that spans a decade, an untrusted inventory will be ignored.

S7. Hybrid and composite blindness

Every hybrid in the benchmark was decomposed into independent findings, and the classical half was reported as a stand-alone vulnerability. When the post-quantum half used an API the tool did not model, a correct hybrid looked entirely classical. Because IR 8547, M-26-15 and SP 800-227 all accommodate hybrids during the transition [8], [9], [16], an analyzer that cannot represent them will over-report migrated systems or, once suppressions are added by hand, under-report broken ones.

S8. No construction checking for post-quantum primitives

Inventory tools stop at naming the algorithm. None checked how a KEM shared secret is used, whether an ephemeral KEM key pair is reused, which ML-DSA variant and context string are used, or whether the state of an LMS or XMSS key is persisted before a signature is released [6], [14], [16]. These are exactly the places where PQC introduces new ways to fail, and the misuse case in our benchmark was reported as a compliant asset.

S9. No agility or migration-blocker analysis

Migration plans must show that systems are cryptographically agile [9], [19], and agility failures are often size assumptions rather than algorithm names: fixed buffers, database columns, message frames or certificate chains sized for RSA or ECDSA. ML-DSA-65 public keys and signatures are 1,952 and 3,309 bytes, and ML-KEM-768 encapsulation keys and ciphertexts are 1,184 and 1,088 bytes [5], [6], so such assumptions break at migration time. No tool looked for them, and measuring agility has been recognized as an open research problem for several years [53].

S10. Weak evidence semantics and fragile tooling

Finally, tool outputs do not support the claims made with them. Grades, readiness scores and exit codes are computed over findings alone, with no statement of which languages, libraries, versions, files and analysis depths were covered, so a clean report cannot be told apart from a blind one. Formats differ, spanning SARIF, CBOM, bespoke JSON, plain text and file-level verdicts, and none reports confidence or an explicit "unknown" class, even though M-26-15 advises treating unidentified asymmetric algorithms as vulnerable [9]. Operational fragility compounds the problem. During this study one scanner's published command was broken, one crashed on a dependency conflict, one older tool crashed on a current Python version, and the most capable PQC inventory plugin requires a SonarQube server. Language coverage is thin as well: CodeQL's experimental quantum queries cover Java, three of the PQC scanners handle only Python, and PQC-aware modeling of C and C++, where OpenSSL and most embedded cryptography live, is limited in CodeQL 2.27.2 to an experimental library that ships a single CBOM-graph query.

VI. Improving Static Analyzers for PQC Compliance Evaluation

The shortcomings share a remedy: separate the collection of evidence from the judgment applied to it, make every layer explicit about what it could not see, and validate each layer against ground truth. Fig. 1 shows a reference architecture built on that principle. The techniques below populate its layers, and each names the shortcomings it addresses.

[embedded content: Fig. 1 · reference architecture for PQC-aware static analysis]

Evidence from five kinds of artifacts is normalized into one CBOM-based asset model; dated policy profiles and PQC-specific checks run only on that model, and every output carries a coverage manifest so that silence has a defined meaning.

A. Separate detection from judgment (S1, S10)

Analyzers should emit facts rather than verdicts: which algorithm, parameter set, primitive, operation and purpose, at which location, with what confidence. CycloneDX CBOM already provides most of this vocabulary, and the algorithm-family registry of version 1.7 gives tools a shared naming scheme [20]. Judgment then becomes a separate, versioned policy layer evaluated against the asset model for a stated profile and date, so that the same inventory can be assessed against NIST IR 8547, CNSA 2.0, Canada's ITSP.40.111 or an organization's own profile [8], [11], [18]. Policies should be written as code with test cases, including dated rules such as the proposed deprecation of 112-bit quantum-vulnerable schemes after 2030, parameter rules such as ML-KEM-1024 only, and the M-26-15 default that unidentified asymmetric algorithms count as vulnerable [9]. Separating the layers also removes the conflation of classical weakness with quantum risk, because each becomes a distinct policy question over the same facts.

B. Maintain versioned, tested API models (S2)

Library models should be first-class, versioned artifacts with release-tracking tests. Each model would declare the library versions it covers, and continuous integration would fail when a new library release adds cryptographic entry points that the model does not map, which can be detected from type stubs, public symbol tables or the library's own tests. Positive recognition rules for ML-KEM, ML-DSA, SLH-DSA, LMS, XMSS and their hybrids should be mandatory. Algorithm identifiers should be resolved through exact registries, such as object identifiers, JCA standard names and CBOM families, rather than substring matching, which is what turned ML-DSA into DSA in four analyzers. CrySL shows that such models can be written declaratively and shared across analyzers [26]; a community-maintained PQC extension of that idea would let small tools inherit current models instead of re-deriving them.

C. Spend semantic depth where migration code needs it (S3, S4)

Three analyses close most of the variant gap at moderate cost. Inter-procedural constant and string propagation resolves algorithm names and key sizes passed through helpers and configuration, as CodeQL's resolution of the properties-file case shows is feasible [25]. Type-based call resolution handles injected modules and wrapper classes. For reflection and dynamic loading, which no practical analyzer resolves soundly [31], [52], the analyzer should report an explicit unresolved cryptography-capable call whenever a dynamically chosen target could reach a cryptographic API, instead of staying silent. Under M-26-15's conservative default, such findings would count as potentially vulnerable until resolved [9].

D. Fuse evidence across artifacts (S4, S5)

Source analysis should be one extractor among several feeding the same asset model. Configuration parsers would cover YAML, properties files, OpenSSL configuration, java.security and infrastructure-as-code, including cloud KMS key specifications. Software composition analysis would annotate each dependency with its cryptographic capabilities, so that a call into requests is linked to the TLS stack it ultimately uses. Container and binary scanners such as CBOMkit-theia would contribute certificates, keys and configuration [33], and runtime or network observation would record the TLS groups actually negotiated. Each asset would carry provenance and confidence, and joins across sources would let a source-level TLS call inherit the groups negotiated in deployment, which resolves the agility paradox of S4.

E. Add PQC-specific construction checks (S7, S8)

The asset model needs a composite type for hybrids that records the classical component, the post-quantum component and the combiner, so that X25519 inside an approved hybrid is evaluated as part of the hybrid [16]. Typestate specifications can then encode the usage rules that SP 800-227 and the signature standards impose. KEM shared secrets should reach symmetric keys only through an approved derivation, ephemeral KEM key pairs should not be reused, ML-DSA calls should use the intended variant and context, and stateful hash-based signing should persist the updated key state before a signature is released [6], [14], [16]. These are data-flow and typestate properties of the kind CrySL-based analyzers already check for classical APIs [26].

F. Measure agility and find migration blockers (S9)

Agility can be approximated statically by classifying how each algorithm choice is bound: as a literal in code, a named constant, external configuration, or protocol negotiation [19], [53]. A second analysis tracks data flow from cryptographic outputs such as signatures, public keys, ciphertexts and certificates into fixed-size sinks, including packed structures, buffers, length checks and schema constants, and flags sizes that cannot hold the post-quantum equivalents [5], [6]. Together the two analyses turn migration readiness from an aspiration into a measurable property of code.

G. Make context and silence explicit (S6, S10)

Each finding should carry reachability, a production-or-test label and a purpose label, and data-lifetime annotations supplied by system owners would allow Mosca-style prioritization of harvest-now-decrypt-later exposure [4]. Every report should include a coverage manifest listing the languages, libraries and versions modeled, the files analyzed, the analysis depth and the unresolved sites, emitted alongside the CBOM and as SARIF for developer workflows [20], [54]. Grades and readiness scores, if retained, should be computed over the manifest as well as the findings, so that a project the tool cannot see receives “unknown” rather than a passing grade. Inventories should be diffed over time, and continuous integration should enforce a ratchet that blocks new quantum-vulnerable usage while legacy findings are migrated.

H. Validate continuously (all)

PQC analyzers need the evaluation infrastructure that classical misuse detection acquired over a decade [12], [45], [46]. That means a shared ground-truth benchmark covering inventory, recognition, hybrids, policy, construction and agility; mutation operators adapted to PQC, such as aliasing, injection, configuration, reflection, wrapping in third-party APIs and splitting into hybrids; differential testing across tools; and published per-category precision and recall for each release. The benchmark accompanying this paper is a seed for such a suite, not a substitute for it.

VII. Discussion, Research Agenda and Threats to Validity

A. What a static report can certify

A static source-code analyzer can contribute evidence to four of the seven compliance obligations of Section II-B, but it cannot certify compliance on its own. The defensible form of a claim is bounded: within a stated coverage manifest, under a named policy profile at a named date, no unresolved findings remain. Migration plans that must name “the methodologies and automated tools used for the cryptographic inventory” [9] should therefore report coverage and analysis depth, not tool names alone, which is consistent with Canada’s advice to combine discovery methodologies [10].

Until analyzers improve, practitioners can reduce risk with current tools. A call-site-precise scanner and a recall-oriented keyword scanner disagree in informative ways, so their differences make a useful review queue. Configuration and container scanning should run beside source scanning. Post-quantum and hybrid code should be tracked in a curated allow-list rather than inferred from tool silence, and letter grades or readiness scores should never be cited as compliance evidence.

B. Research agenda

Several open problems follow from the taxonomy. The first is API-model maintenance at ecosystem scale: synthesizing and testing cryptographic API models from type stubs, documentation and library test suites, possibly with machine assistance, but always with deterministic verification. The second is static and dynamic fusion that binds source-level findings to the behavior negotiated at run time, so that a TLS call site inherits the groups a deployment actually uses. The third is a formal semantics for dated, composable compliance policies over CBOM data, including hybrids and composite signatures. The fourth is agility metrics validated against real migrations, so that static measures of binding and size assumptions can predict migration effort. The fifth is benchmark infrastructure: a shared PQC benchmark with ground truth and mutation operators, extended to C, C++, Rust, Go and firmware. The sixth concerns embedded and IoT systems, whose long service lives, constrained message sizes and firmware-signing requirements under CNSA 2.0 [11] make them the hardest targets and the least served by current tools. Language models may help with purpose and context labeling, but their output would need the same coverage and ground-truth discipline proposed here, or they would add confident-sounding noise to inventories.

C. Threats to validity

Internal validity is limited by our own construction of ground truth and by the normalization of heterogeneous outputs into algorithm families. We mitigated the second by manually reviewing every post-quantum, hybrid and control cell and by publishing raw outputs and scripts. Several case files mention algorithm names in docstrings, which favors text-matching tools; reporting recall

on code lines controls for this. Construct validity is limited because a case counts as detected when the expected family appears anywhere in its files, which is generous to the tools and ignores location precision. External validity is the main limitation. Forty-two small synthetic cases, mostly in Python, cannot represent real code bases, so the results are evidence of failure modes rather than rankings. Several tools are young and change quickly, and the results describe the versions available on 15 October 2026 under default configurations; tuned rules could change individual outcomes. We could not execute sonar-cryptography or any commercial discovery product, and our workarounds for cryptohound and pqc-posture may differ from their authors' intended use.

VIII. Conclusion

Open-source static analyzers are being asked to underpin PQC compliance claims at the moment mandates require inventories: Canadian departments have owed migration plans since April 2026, and United States civilian agencies must submit theirs, naming their inventory tools, within 120 days of 24 June 2026 [9], [10]. Our review and micro-benchmark show why that reliance is premature. General-purpose SAST measures classical strength and is nearly blind to quantum exposure. PQC-branded scanners find canonical usages but are brittle under semantics-preserving change, embed a single implicit policy, cannot credit post-quantum or hybrid code reliably, sometimes mislabel ML-DSA as DSA, and never say what they could not see. No tool examined checks PQC construction or crypto-agility.

The remedy is architectural rather than a matter of more rules: separate evidence from judgment, maintain tested API models, fuse evidence across artifacts, add PQC-specific construction and agility checks, and make coverage explicit, all validated against shared ground truth. Until analyzers work this way, a clean static report should be read as absence of evidence, not as evidence of absence.

References

[1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997. doi:10.1137/S0097539795293172

[2] C. Gidney and M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 5, p. 433, 2021. doi:10.22331/q-2021-04-15-433

[3] C. Gidney, “How to factor 2048 bit RSA integers with less than a million noisy qubits,” arXiv:2505.15917, May 2025. arxiv.org/abs/2505.15917

[4] M. Mosca, “Cybersecurity in an era with quantum computers: Will we be ready?” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[5] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, Aug. 2024. doi:10.6028/NIST.FIPS.203

[6] NIST, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, Aug. 2024. doi:10.6028/NIST.FIPS.204

[7] NIST, “Stateless Hash-Based Digital Signature Standard,” FIPS 205, Aug. 2024. doi:10.6028/NIST.FIPS.205

[8] D. Moody, R. Perlner, A. Regenscheid, A. Robinson and D. Cooper, “Transition to Post-Quantum Cryptography Standards,” NIST IR 8547 (Initial Public Draft), Nov. 2024. csrc.nist.gov/pubs/ir/8547/ipd

[9] Office of Management and Budget, “Execution of the Migration to Post-Quantum Cryptography,” Memorandum M-26-15, Jun. 24, 2026. whitehouse.gov

[10] Canadian Centre for Cyber Security, “Roadmap for the migration to post-quantum cryptography for the Government of Canada (ITSM.40.001),” Jun. 2025. cyber.gc.ca

[11] National Security Agency, “Announcing the Commercial National Security Algorithm Suite 2.0,” Cybersecurity Advisory, Sep. 2022, updated May 2025. media.defense.gov

[12] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshyvanyk and A. Nadkarni, “Why crypto-detectors fail: A systematic evaluation of cryptographic misuse detection techniques,” in *Proc. IEEE Symp. Security and Privacy*, 2022.

[13] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th ACM Symp. Theory of Computing*, 1996, pp. 212–219. doi:10.1145/237814.237866

[14] D. A. Cooper, D. C. Apon, Q. H. Dang, M. S. Davidson, M. J. Dworkin and C. A. Miller, “Recommendation for Stateful Hash-Based Signature Schemes,” NIST SP 800-208, Oct. 2020. doi:10.6028/NIST.SP.800-208

[15] SecurityWeek, “NIST announces HQC as fifth standardized post quantum algorithm,” Mar. 2025. securityweek.com

[16] G. Alagic, E. Barker, L. Chen, D. Moody, A. Robinson, H. Silberg and N. Waller, “Recommendations for Key-Encapsulation Mechanisms,” NIST SP 800-227, Sep. 2025. doi:10.6028/NIST.SP.800-227

[17] PostQuantum.com, analysis of the NSA CNSA 2.0 FAQ version 2.1 update (secondary source), 2025. postquantum.com

[18] Canadian Centre for Cyber Security, “Cryptographic algorithms for UNCLASSIFIED, PROTECTED A, and PROTECTED B information (ITSP.40.111).” cyber.gc.ca

[19] NIST, “Considerations for Achieving Cryptographic Agility,” NIST CSWP 39. csrc.nist.gov

[20] OWASP Foundation and Ecma TC54, "CycloneDX Bill of Materials Specification (ECMA-424)," v1.6, Apr. 2024, and v1.7, Oct. 2025. github.com/CycloneDX/specification; v1.7 release notes

[21] NIST NCCoE, "Migration to Post-Quantum Cryptography. Quantum Readiness: Cryptographic Discovery," NIST SP 1800-38B (Preliminary Draft), Dec. 2023. nccoe.nist.gov

[22] Wind River, "Crypto Detector," source repository, commit d6156a8, Jan. 2020. github.com/Wind-River/crypto-detector

[23] PyCQA, "Bandit," version 1.9.4. github.com/PyCQA/bandit

[24] Semgrep, Inc., "Semgrep," version 1.181.0. github.com/semgrep/semgrep

[25] P. Avgustinov, O. de Moor, M. Peyton Jones and M. Schäfer, "QL: Object-oriented queries on relational data," in *Proc. ECOOP*, 2016, LIPIcs vol. 56. doi:10.4230/LIPIcs.ECOOP.2016.2; GitHub CodeQL CLI 2.27.2.

[26] S. Krüger, J. Späth, K. Ali, E. Bodden and M. Mezini, "CrySL: An extensible approach to validating the correct usage of cryptographic APIs," in *Proc. ECOOP*, 2018, LIPIcs vol. 109. doi:10.4230/LIPIcs.ECOOP.2018.10

[27] R. E. Strom and S. Yemini, "Typestate: A programming language concept for enhancing software reliability," *IEEE Trans. Softw. Eng.*, vol. SE-12, no. 1, pp. 157–171, 1986.

[28] T. Reps, S. Horwitz and M. Sagiv, "Precise interprocedural dataflow analysis via graph reachability," in *Proc. 22nd ACM Symp. Principles of Programming Languages*, 1995, pp. 49–61.

[29] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM Symp. Principles of Programming Languages*, 1977, pp. 238–252. doi:10.1145/512950.512973

[30] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Trans. Amer. Math. Soc.*, vol. 74, no. 2, pp. 358–366, 1953.

[31] B. Livshits et al., "In defense of soundiness: A manifesto," *Commun. ACM*, vol. 58, no. 2, pp. 44–46, 2015. doi:10.1145/2644805

[32] CBOMkit, "Sonar Cryptography Plugin (CBOMkit-hyperion)," source repository, commit 58eea48, Oct. 8, 2026. github.com/cbomkit/sonar-cryptography

[33] IBM Research and PQCA, "CBOMkit" and "CBOMkit-theia," source repositories. github.com/IBM/cbomkit; github.com/cbomkit/cbomkit-theia

[34] GitHub, "Crypto: Add Java cryptographic check queries," CodeQL pull request 20568, and the experimental quantum library in CodeQL CLI 2.27.2 (quantum pack 0.0.37). github.com/github/codeql/pull/20568

[35] Semgrep, Inc., "semgrep-rules," Semgrep Rules License v1.0, develop branch fetched Oct. 15, 2026. github.com/semgrep/semgrep-rules

[36] Python Cryptographic Authority, "pyca/cryptography," version 50.0.2. cryptography.io

[37] qproof, version 0.4.0. pypi.org/project/qproof

[38] cryptohound, version 0.1.0. pypi.org/project/cryptohound

[39] pqc-audit (PQC Scanner), version 0.4.1. pypi.org/project/pqc-audit

[40] pqready, version 0.2.2. pypi.org/project/pqready

[41] crypto-scanner, version 0.1.2. pypi.org/project/crypto-scanner

[42] pqc-posture (PQC Posture Scanner), version 0.4.0, project homepage as declared in the package metadata. github.com/Miles0sage/quantum-mcp

[43] M. Egele, D. Brumley, Y. Fratantonio and C. Kruegel, "An empirical study of cryptographic misuse in Android applications," in *Proc. ACM CCS*, 2013, pp. 73–84. doi:10.1145/2508859.2516693

[44] S. Rahaman et al., "CryptoGuard: High precision detection of cryptographic vulnerabilities in massive-sized Java projects," in *Proc. ACM CCS*, 2019, pp. 2455–2472. doi:10.1145/3319535.3345659

[45] S. Afrose, S. Rahaman and D. Yao, "CryptoAPI-Bench: A comprehensive benchmark on Java cryptographic API misuses," in *Proc. IEEE Secure Development Conf. (SecDev)*, 2019.

[46] M. Schlichtig, A.-K. Wickert, S. Krüger, E. Bodden and M. Mezini, "CamBench: Cryptographic API misuse detection tool benchmark suite," in *Proc. Mining Software Repositories (MSR)*, 2022.

[47] A.-K. Wickert, L. Baumgärtner, F. Breitfelder and M. Mezini, "Python crypto misuses in the wild," in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2021.

[48] OpenJDK, "JEP 496: Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism," JDK 24, 2025. openjdk.org/jeps/

[49] OpenJDK, "JEP 497: Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm," JDK 24, 2025. openjdk.org/jeps/497

[50] B. Johnson, Y. Song, E. Murphy-Hill and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. Int. Conf. Software Engineering (ICSE)*, 2013, pp. 672–681.

[51] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon and C. Jaspan, "Lessons from building static analysis tools at Google," *Commun. ACM*, vol. 61, no. 4, pp. 58–66, 2018.

[52] D. Landman, A. Serebrenik and J. J. Vinju, "Challenges for static analysis of Java reflection: Literature review and empirical study," in *Proc. Int. Conf. Software Engineering (ICSE)*, 2017, pp. 507–518.

[53] D. Ott, C. Peikert et al., "Identifying research challenges in post quantum cryptography migration and cryptographic agility," Computing Community Consortium workshop report, arXiv:1909.07353, 2019.

[54] OASIS, "Static Analysis Results Interchange Format (SARIF) Version 2.1.0," OASIS Standard, 2020.

Appendix

Table A1. Per-tool results on the micro-benchmark (15 October 2026, default settings)

Quantum-vulnerable (QV) recall counts findings on code lines only. "n/a" marks a language the tool does not analyze. False positives are counted over the three no-cryptography controls (e1, e2, g1).

Tool and version	Family	Python QV found (of 17)	Java QV found (of 4)	Stand-alone PQC credited (of 7)	ML-DSA read as DSA	Hybrid X25519 reported alone	Code-line false positives (of 3)	Notable behavior
qproof 0.4.0	PQC scanner	13	3	0	Yes	Yes	2	Found all four third-party cases; marked SHA-256 "no change needed"
Crypto Detector, keyword method	Keyword scanner	12	3	0	Yes	Yes	2	Only tool to find the YAML-configured RSA; API method crashes on Python 3.12
cryptohound 0.1.0	PQC scanner	11	n/a	0	No	Yes	0	8 of 8 canonical, 2 of 5 variants; ran only with a working Semgrep on the path
pqc-audit 0.4.1	PQC scanner	10	n/a	3	No	Yes	0	Credited liboqs names only; listed the KEM misuse as a compliant asset
crypto-scanner 0.1.2	PQC scanner	10	3	5	Yes (JCA case)	Yes	0	Generic "post-quantum" label; rated SHA-256 a medium risk
pqc-posture 0.4.0	PQC scanner	9	2	0	No	No	0	CLI broken, run through its API; lowered test-code severity; reported TLS 1.3 as TLS 1.0/1.1

Tool and version	Family	Python QV found (of 17)	Java QV found (of 4)	Stand-alone PQC credited (of 7)	ML-DSA read as DSA	Hybrid X25519 reported alone	Code-line false positives (of 3)	Notable behavior
pqready 0.2.2	PQC scanner	6	n/a	0	No	No	0	File-level findings without line numbers
Bandit 1.9.4	General SAST	2	n/a	0	No	No	0	Only the RSA-1024 key-size rule and a PyCrypto deprecation rule fired
Semgrep 1.181.0, community rules	General SAST	1	0	0	No	No	0	RSA-1024 only; flagged the non-security SHA-1
CodeQL 2.27.2, security-extended	General SAST	1	0	0	No	No	0	Reported ML-KEM-768 and ML-DSA as potentially weak; flagged TLS 1.0 in the default context
CodeQL 2.27.2, experimental quantum	Experimental PQC inventory	n/a	4	1	Yes	n/a	0	Resolved RSA from a properties file; labelled ML-KEM-768 as key agreement

Table A2. Benchmark cases and ground truth

Case	Files	Pattern	Ground truth
a1	a1_rsa_keygen_2048.py	RSA-2048 key generation and PSS signature (pyca)	QV: RSA
a2	a2_rsa_keygen_1024.py	RSA-1024 key generation	QV: RSA, also classically weak
a3	a3_ecdsa_p256_sign.py	ECDSA over P-256	QV: ECDSA
a4	a4_x25519_exchange.py	X25519 agreement with HKDF	QV: X25519
a5	a5_ed25519_sign.py	Ed25519 signature	QV: EdDSA
a6	a6_ffdh_exchange.py	Finite-field Diffie-Hellman, 2048-bit	QV: DH
a7	a7_rsa_oaep_loaded_key.py	RSA-OAEP with a loaded public key	QV: RSA
a8	a8_pycryptodome_rsa.py	RSA-3072 with PKCS#1 v1.5 (PyCryptodome)	QV: RSA
b1	b1_rsa_aliased_import.py	a1 with an aliased module and positional arguments	QV: RSA
b2	b2_main.py, b2_keyutil.py	Algorithm module injected into a generic helper	QV: RSA
b3	b3_config_driven.py, b3_crypto.yaml	Algorithm family read from YAML	QV: RSA, from configuration
b4	b4_curve_from_env.py	Curve chosen by environment variable	QV: ECDSA
b5	b5_reflective_import.py	Module name built at run time, reflective call	QV: RSA
c1	c1_jwt_rs256.py	PyJWT with RS256	QV: RSA
c2	c2_paramiko_ssh_keys.py	Paramiko RSA and ECDSA host keys	QV: RSA and ECDSA
c3	c3_tls_default_context.py	Default TLS context	Deployment-dependent key exchange
c4	c4_openssl_cli.py	OpenSSL CLI generating RSA-3072	QV: RSA
c5	c5_requests_https.py	HTTPS through requests	Deployment-dependent, transitive TLS
c6	c6_cloud_kms_keyspec.py	Cloud KMS key with an RSA_3072 key specification	QV: RSA
d1	d1_mlkem768_liboqs.py	ML-KEM-768 (liboqs)	PQC to credit
d2	d2_mldsa65_liboqs.py	ML-DSA-65 (liboqs)	PQC to credit
d3	d3_hybrid_x25519_mlkem768.py	X25519 plus ML-KEM-768 combined with	Hybrid

Case	Files	Pattern	Ground truth
		HKDF (liboqs)	
d4	d4_mlkem512_param.py	ML-KEM-512	PQC to credit under NIST, not under CNSA 2.0
d5	d5_kem_secret_misuse.py	KEM secret used directly as an AES key; key pair reused	PQC construction defect
d6	d6_pyca_mlkem768.py	ML-KEM-768 (pyca/cryptography native)	PQC to credit
d7	d7_pyca_mldsa65.py	ML-DSA-65 with a context string (pyca/ cryptography native)	PQC to credit
d8	d8_hybrid_pyca.py	X25519 plus ML-KEM-768 combined with HKDF (pyca/cryptography)	Hybrid
e1	e1_comments_only.py	Algorithm names in comments and strings only	No cryptography
e2	e2_ecc_reed_solomon.py	Error-correcting code with identifiers ecc, dh and dsa	No cryptography
e3	e3_sha1_nonsecurity.py	SHA-1 with usedforsecurity=False	Not quantum-relevant
e4	e4_unreachable_rsa.py	RSA in unreachable code	QV, unreachable
t1	tests/test_rsa_fixture.py	RSA test fixture	QV, test-only
f1	f1_aes128_gcm.py	AES-128-GCM	Acceptable under NIST, not CNSA 2.0
f2	f2_sha256_digest.py	SHA-256 digest	Acceptable under NIST, not CNSA 2.0
g1	g1_fixed_signature_buffer.py	256-byte signature and 294-byte key framing	Migration blocker, no cryptography
j1	J1RsaSign.java	RSA-2048 with SHA256withRSA	QV: RSA
j2	J2EcdsaSign.java	ECDSA over secp256r1	QV: ECDSA
j3	J3X25519Agree.java	X25519 key agreement	QV: X25519
j4	J4MlKemJca.java	ML-KEM-768 through the JCA KEM API	PQC to credit
j5	J5MlDsaJca.java	ML-DSA-65 through JCA	PQC to credit
j6	J6ConfigDriven.java, crypto.properties	Algorithms read from a properties file	QV: RSA, from configuration
j7	J7HybridTlsGroups.java	TLS 1.3 socket with platform default groups	Deployment-dependent key exchange

The raw tool outputs, the full case-by-tool matrix of reported algorithm families and the scoring scripts are distributed with this paper as pqc-sast-bench.zip.